PROJECT COMPLETION REPORT

Autonomous Data Science Co-Pilot

An Agentic AI System for Natural-Language Data Analysis

Summer Internship 2025

Project Type: Agentic AI / Data Engineering / Full-Stack Python

Prepared by: Tanishk Sharma

1. Executive Summary

The Autonomous Data Science Co-Pilot is an agentic AI system that lets a non-technical user upload a raw dataset, ask a question in plain English, and receive a finished analytical artefact — a chart, a statistic, or a written insight — without writing any code. This directly implements the brief's core objective of moving "from Text-Out to Action-Out."

The delivered system consists of a Streamlit web application, a LangChain-orchestrated agent backed by Google Gemini models, a security-hardened local code-execution sandbox, and a ChromaDB-based retrieval-augmented self-healing loop. Beyond the baseline brief, the implementation adds a full business-analytics layer (KPI dashboards, data-health scoring, outlier and anomaly detection, KMeans cohort segmentation, time-series forecasting, and automated PDF reporting), multi-tenant user isolation, and a two-stage self-correction mechanism that distinguishes technical execution errors from cases where the code ran cleanly but did not answer the question asked.

All five use cases specified in the project brief — sales dashboarding, data quality auditing, trend analysis, cohort analysis, and ad-hoc natural-language querying — are implemented and traceable to specific modules, detailed in Section 5.

2. Problem Statement & Objectives

Business teams routinely sit on raw data — CSVs, Excel exports, JSON dumps — without the in-house expertise to extract insight from it. The brief tasked the project with building an AI agent that behaves like a junior data analyst: independently writing Python/Pandas code against an uploaded file, executing that code in a sandboxed environment, self-correcting failures using retrieval-augmented generation over documentation, and returning professional charts and insights with zero coding required from the end user.

2.1 Key Innovation

The differentiator called out in the brief — delivering a finished artefact rather than instructions — is realized end-to-end: the agent's output is not a code snippet for the user to run, but an executed chart, a validated statistic, and a business-facing written summary, generated and verified automatically before being shown to the user.

3. System Architecture

The pipeline follows the brief's recommended flow — upload, generate, execute, self-correct, deliver — implemented as a closed loop between the agent, the sandbox, and two independent correction paths:

Stage	Component	Responsibility
User Upload & Query	Streamlit UI (app.py)	Accepts CSV / Excel / JSON / Parquet / SQLite / TSV / ZIP; profiles the dataset and captures the natural-language question.
Agent Orchestration	DataScienceAgent (agent.py)	Builds the prompt (schema, data summary, conversation history, question), calls the Gemini LLM via LangChain, extracts generated code.
Code Execution	Sandbox (sandbox.py)	Runs generated code in an isolated subprocess with a runtime security shield; captures stdout/stderr/exit status and generated files.
Self-Healing (technical)	RAGPipeline (rag.py) + ChromaDB	On a real execution error, retrieves relevant documentation chunks (MMR search) and feeds them back into a correction prompt.

Stage	Component	Responsibility
Self-Healing (semantic)	Validation LLM pass + VALIDATION_CORRECTION_PROMPT	A second LLM call checks the executed output actually answers the question; mismatches are corrected via a dedicated realignment prompt rather than the documentation-error path.
Business Analytics Layer	business_analyst.py	KPI extraction, health scoring, outlier/anomaly detection, KMeans cohorts, forecasting, PDF report generation — independent of the LLM.

Flow: User Upload & Query → Streamlit UI → DataScienceAgent → Gemini LLM → Sandbox execution. On failure, the sandbox routes back through the RAG pipeline (technical errors) or the validation-correction prompt (semantic mismatches) to the LLM for another attempt, up to three retries, before the result is returned to the UI.

4. Technology Stack: Brief vs. Implementation

Layer	Brief Recommendation	Implemented	Status
Frontend / UI	Streamlit (Gradio)	Streamlit 1.49.1	*Complete*
LLM / Agent	OpenAI GPT-4o or open-source via LangChain / LlamaIndex	Google Gemini 3.1 / 3.5 family via LangChain (langchain-google-genai)	*Complete — provider substituted, orchestration framework matches*
Code Execution	Python subprocess sandbox (restricted exec environment)	Custom Sandbox class: subprocess isolation, blocked network/fork/subprocess/ctypes, workspace-scoped filesystem access, path-traversal shielding	*Complete — exceeds baseline*
RAG Pipeline	FAISS or ChromaDB + documentation corpus	ChromaDB with MMR retrieval + HuggingFace all-MiniLM-L6-v2 embeddings over a curated Pandas/Plotly/scikit-learn error corpus	*Complete — faiss-cpu retained as a dependency but ChromaDB is the active store*
Data Processing	Pandas, NumPy, OpenPyXL, json	Pandas, NumPy, OpenPyXL, SciPy, scikit-learn, statsmodels, PyArrow (Parquet), sqlite3	*Complete — exceeds baseline*
Visualisation	Matplotlib, Seaborn, Plotly	Plotly Express (primary, interactive HTML), Matplotlib (Agg backend fallback), Seaborn	*Complete*

5. Use Case Coverage

All five use cases specified in Section 3 of the brief are implemented and traceable to a specific UI tab or code path:

#	Use Case	Brief Description	Implementation
1	**Sales Dashboard**	Upload a monthly sales CSV — get an instant revenue-by-region bar chart.	Business Dashboard tab: auto-detects region/value columns and renders a "Total Sales by Region" Plotly bar chart, plus a monthly trend line.

#	Use Case	Brief Description	Implementation
2	**Data Quality Audit**	Detect missing values, outliers, and duplicates; produce a cleanliness report.	Data Quality & Profiling tab (health score, missing/duplicate counts, column-type breakdown) + Outliers & Anomalies tab (IQR/Z-score outliers, spike/dip detection).
3	**Trend Analysis**	Ask "Is my traffic growing?" over a time-series CSV and receive a trend line with key observations.	Answerable via the natural-language query box (agent-generated Plotly trend code) and via the dedicated 3-month moving-average / linear-regression forecasting tool.
4	**Cohort Analysis**	Auto-segment customers by spend or activity and produce grouped visualisations.	KMeans Cohort Segmentation (2D/3D) in the Outliers & Anomalies tab, with adjustable cluster count and a Plotly scatter visualisation.
5	**Ad-hoc Queries**	Founders query operational data without writing a single line of code.	Primary chat-style "What would you like to know or analyze?" input, routed through DataScienceAgent end-to-end.

6. Implementation Detail by Module

6.1 Frontend — app.py

A single-page Streamlit application organized into a sidebar (user/project isolation, model selection, API key entry, debug mode) and four content tabs:

- **Data Quality & Profiling:** health score, KPI cards, missing/duplicate counts, column-type breakdown, statistical highlights (skew, kurtosis, extended statistics), and auto-recommended visualisations.
- **Outliers & Anomalies:** IQR/Z-score outlier detection, spike/dip anomaly detection, and interactive KMeans cohort segmentation.
- **Business Dashboard:** auto-generated revenue-by-region, monthly trend, rating-distribution, and correlation-heatmap charts, plus interactive time-series forecasting.
- **Report & Downloads:** one-click executive PDF report, session-level Markdown report export, and cleaned-dataset CSV download.

The application supports CSV, Excel, JSON, Parquet, SQLite, TSV, and ZIP uploads, with guardrails on file size (50MB) and row count (500k rows), and automatically de-duplicates column names before analysis.

6.2 Agent Orchestration — agent.py

The DataScienceAgent class drives the end-to-end request cycle:

- Loads per-project conversational memory (last 4 interactions) for multi-step operations (e.g. "Clean the data" → "Now build a regression model").
- Inspects the dataset's actual columns and injects them into the prompt, alongside a reusable find_column() helper the generated code must use for schema-agnostic column mapping.
- Extracts the generated Python code block and executes it via the Sandbox.
- On success, runs a second, independent LLM validation pass to confirm the output genuinely answers the question asked.

- Retries up to three times, routing technical execution errors through the RAG-grounded correction prompt, and semantic validation mismatches through a dedicated realignment prompt — avoiding wasted retries on irrelevant documentation lookups for non-technical failures.
- Classifies infrastructure-level failures (sandbox/security-shield errors, disallowed imports) separately so the loop does not keep retrying an unwinnable case.

6.3 Execution Sandbox — sandbox.py

Generated code never runs in the main process. Each execution spins up an isolated subprocess with a runtime "security shield" injected ahead of the user code, enforcing:

- No subprocess spawning, forking, or multiprocessing (prevents sandbox-escape via child processes).
- No outbound network connections (socket layer blocked, preventing data exfiltration).
- Restricted native-library loading via ctypes — bare system library names and libraries resolved from within the Python environment are permitted (required for NumPy/SciPy/scikit-learn to initialize), while loads from arbitrary external paths are denied.
- File writes, renames, deletions, and permission changes are restricted strictly to the project's own workspace folder.
- Per-run output versioning: generated files are moved into timestamped folders under the project's outputs directory.

As documented in the README, this is a Python-level runtime guardrail, not OS-level isolation; the project explicitly flags that Docker/gVisor/nsjail-style sandboxing would be required before running fully untrusted or adversarial input in production.

6.4 Self-Healing RAG Pipeline — rag.py

A ChromaDB vector store, embedded with the HuggingFace all-MiniLM-L6-v2 sentence-transformer model, indexes a curated documentation corpus covering common Pandas errors and operations, Pandas KeyError patterns, Plotly Express charting errors, and scikit-learn modelling errors. Retrieval uses Maximal Marginal Relevance (MMR) search to return diverse, relevant chunks, which are injected into the correction prompt alongside the failing code and error message.

6.5 Business Analyst Module — business_analyst.py

A deterministic (non-LLM) analytics layer supplying the dashboard tabs directly from Pandas/Scikit-learn, independent of the agent:

- profile_business_kpis / get_health_score — automatic KPI extraction and a graded dataset health score with actionable recommendations.
- detect_outliers / detect_anomalies — IQR and Z-score based outlier detection, plus spike/dip anomaly detection over numeric fields.
- get_expanded_statistics — variance, coefficient of variation, range, IQR, and percentile statistics.
- recommend_visualizations / suggest_questions — heuristic suggestions for charts and follow-up natural-language prompts.
- run_forecast — 3-month moving-average and linear-regression forecasting rendered as a Plotly trend chart.
- run_cohort_segmentation — StandardScaler + KMeans clustering with 2D/3D Plotly visualisation and cluster summaries.
- generate_pdf_report — an FPDF-based executive PDF export of the dataset's business summary.

7. Security Hardening

Security guardrails are enforced at the Python runtime level inside the execution subprocess:

- **Process control:** subprocess spawning, multiprocessing, forking, and module reloading are disabled to close off sandbox-escape vectors.
- **Network isolation:** outbound socket connections are blocked at the socket layer, covering urllib, requests, and raw sockets.
- **Native code loading:** ctypes-based native library loading is allow-listed rather than blanket-blocked, so legitimate packages (NumPy, SciPy, scikit-learn) can load their own bundled runtime libraries on import, while loads from outside the Python environment are denied.
- **Filesystem protection:** writes, renames, deletions, and shutil-based mutations are restricted strictly to the active project's workspace folder; low-level os.open descriptor calls are checked the same way.
- **Path traversal shielding:** user ID and project name inputs are whitelisted to [A-Za-z0-9_-]+ and every resolved path is verified against an absolute-path prefix check before use.

The project's own documentation is explicit that these are runtime guardrails, not a substitute for OS-level isolation (Docker, gVisor, or nsjail) in a production deployment handling untrusted or adversarial input — an accurate and appropriately conservative security posture for the current stage of the project.

8. Deliverables Checklist

Deliverable (per Brief §6)	Status
Web-based UI (Streamlit) — upload data, ask questions in plain English, get instant charts	**Complete**
Autonomous agent that writes, executes, and self-corrects Python/Pandas code	**Complete**
RAG pipeline over official Python and Pandas documentation	**Complete**
Support for CSV, Excel (.xlsx), and JSON file formats	**Complete — exceeds requirement (adds Parquet, SQLite, TSV, ZIP)**
GitHub repository with clean code, README, and setup instructions	**Complete — README.md includes architecture, security notes, and install/run instructions**
Final demo covering at least 5 end-to-end use case demonstrations	**Complete video of demo in README.MD**

9. Enhancements Beyond the Original Brief

Several capabilities were added on top of the brief's minimum scope:

- Multi-tenant user isolation with per-User-ID workspaces, preventing data leakage across users.
- Persistent, per-project conversational memory enabling multi-step operations across turns.
- A dynamic schema-mapping helper (find_column) that resolves business concepts to actual column names via case-insensitive and synonym matching, so generated code never hardcodes column names.
- A second, independent LLM validation pass after execution, catching cases where code runs cleanly but silently answers a different question than the one asked.

- A dedicated correction path for validation (semantic) failures, separate from the RAG-grounded correction path for technical execution errors — avoiding wasted retries on irrelevant documentation lookups.
- A full business-analytics layer independent of the LLM: health scoring, outlier/anomaly detection, KMeans cohort segmentation, time-series forecasting, and automated PDF reporting.
- Defense-in-depth sandbox hardening beyond a "restricted exec environment": process, network, native-library, and filesystem controls, plus regex- and path-prefix-based traversal protection.

10. Known Limitations & Recommendations

- **Sandbox isolation:** current guardrails operate at the Python runtime level; OS-level isolation (Docker, gVisor, or nsjail) is recommended before processing fully untrusted or adversarial uploaded content in production, as the project's own README already notes.
- **Documentation corpus:** the RAG corpus is currently a small, hand-authored set of common error patterns; expanding it with the full official Pandas/Plotly/scikit-learn documentation would broaden self-healing coverage for less common errors.
- **LLM provider:** the brief suggested OpenAI GPT-4o; the implementation uses Google Gemini via LangChain. This is a reasonable substitution under the brief's "Good to Have" prerequisite of "any LLM API," but should be called out explicitly if GPT-4o compatibility is a hard requirement.
- **Demo coverage:** all five brief use cases are implemented in code; a formal, recorded end-to-end demo walkthrough (Brief §6, final bullet) remains an outstanding action item.

11. Conclusion

The Autonomous Data Science Co-Pilot fulfils the core mandate of the brief: a non-technical user can upload a dataset, ask a question in plain English, and receive an executed, validated analytical artefact without writing code. The implementation matches or exceeds the brief's recommended technology stack across every layer, covers all five specified use cases, and adds a substantial business-analytics and security-hardening layer beyond the original scope. The remaining gaps — a broader documentation corpus for the RAG pipeline and a formal recorded demo — are narrow, well-defined, and do not affect the system's core functionality.